

LINUX FOUNDATIONS · HANDS-ON COURSE

Linux Foundations

A hands-on introduction to operating Linux —
from the shell to systemd.

10 modules

10 labs

Ubuntu 24.04

kernel · files · pipelines · packages · permissions
ssh · journal · systemd · processes · drop-ins

learner@linux-lab-02 — bash

```
learner@lab:~$ uname -r  
6.17.0-1026-nvidia
```

```
learner@lab:~$ systemctl is-active ssh  
active
```

```
learner@lab:~$ echo $?  
0
```

```
learner@lab:~$ █
```

Course agenda

01	System boundaries & the terminal what am I running? reading a command line	lab 15m
02	Filesystem, paths & links navigating the tree; hard links vs symlinks	lab 25m
03	Shell streams, pipelines & text stdin, stdout, stderr; turning logs into answers	lab 20m
04	Packages & software provenance installing with apt; proving origin with dpkg	lab 20m
05	Users, groups & permissions reading ls -l; repairing access safely	lab 30m
06	SSH: host trust & user keys verified fingerprints; key-based login	lab 25m
07	Logs & the systemd journal scoping journalctl; finding the causal event	lab 20m
08	systemd services & dependencies diagnosing units; writing your own service	lab 25m
09	Processes, sockets & behavior tracing PID, port, and HTTP with ps, ss, curl	lab 15m
10	Unit layering & drop-in overrides changing services safely; reload vs restart	lab 15m

Ten laboratory exercises · 210 minutes of supervised hands-on work · individual Ubuntu 24.04 environments.

Laboratory time per module is fixed; any schedule adjustment reduces lecture time only.

01

System boundaries and terminal survival

Which system am I operating, and how does the shell turn text into work?

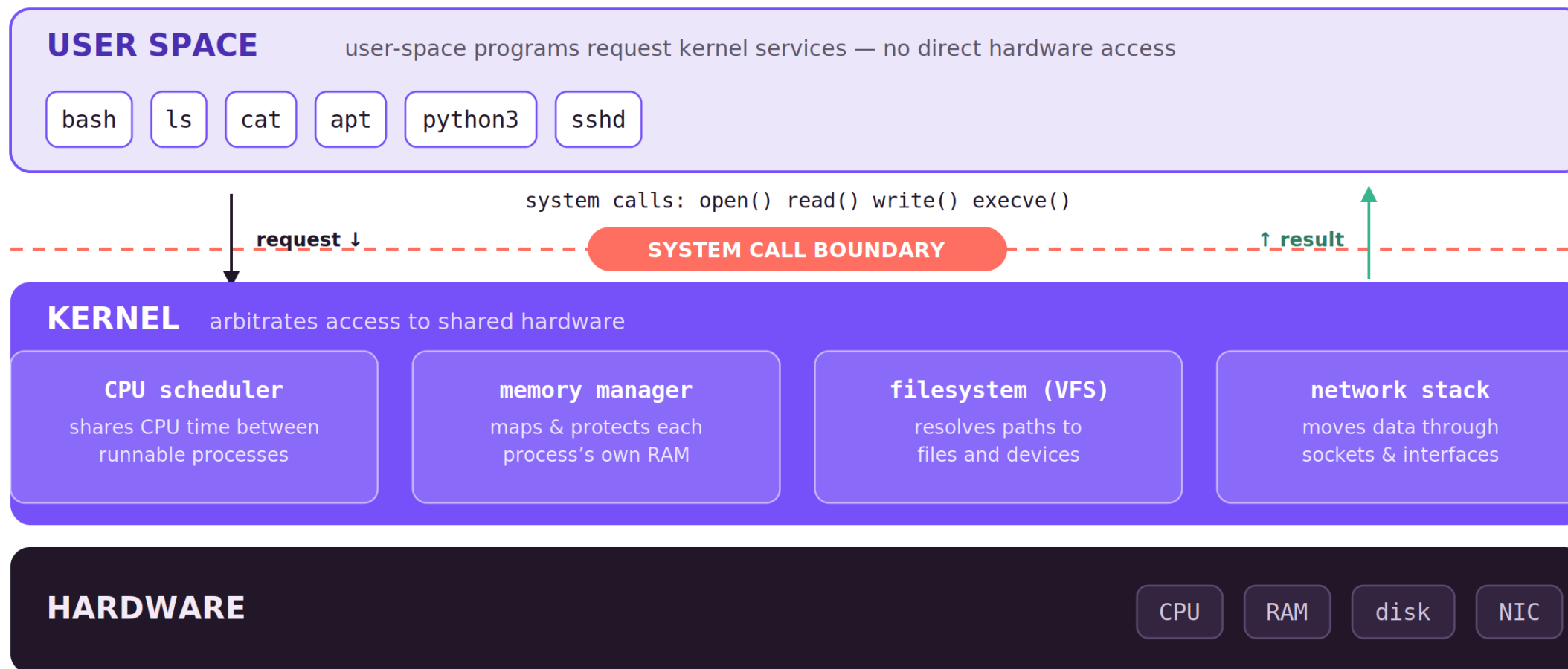
General Linux + Bash · Ubuntu 24.04 lab

What am I actually running?

Linux, distribution, virtual machine, container — different layers that answer different questions.

- Linux is strictly the kernel. A distribution such as Ubuntu packages the user-space tools, services, and defaults around it.
- A virtual machine boots its own kernel; a container has a private user space but shares the host's kernel.

The system in layers



Your command travels down as system calls; bytes and an exit status come back up.

The kernel arbitrates shared resources

Applications request these services; they do not control hardware directly.

- Schedules CPU time between runnable processes
- Maps virtual memory and enforces protection
- Resolves filesystem and device operations
- Moves network data through sockets and interfaces

A system call crosses the user–kernel boundary

A command can fail in the shell, the program, or the kernel-facing operation.

- The shell starts a user-space program such as cat
- cat asks the kernel to open and read a path
- The kernel checks credentials and resolves the filesystem object
- Bytes return to cat, which writes them to standard output

Distribution, VM, and container describe different layers

Record both user-space and kernel facts before following external instructions.

- A distribution curates user space, packages, defaults, and support
- A virtual machine has virtual hardware and its own guest kernel
- A system container has isolated user space but shares the host kernel
- `/etc/os-release` and `uname` answer different layer questions

How the shell reads a line

You type text; the shell prepares it before any program runs.

- The prompt only reports context. The first word names a program or built-in; the rest of the line is its arguments.
- Before running, Bash substitutes `$VAR` and `$(command)` and applies quotes; afterwards it records an exit status, where 0 means success.

Read a command from left to right

```
printf 'USER=%s\nKERNEL=%s\n' \  
"$USER" "$(uname -r)" > ~/profile.txt
```

- printf takes a format then two arguments; %s slots each value in.
- Double quotes keep each value one argument, even with spaces.
- > replaces the file (>> appends); the shell opens it, not printf.
- \$(uname -r) runs first — substituted before printf even starts.

These commands are tools for specific questions

The lab guide repeats syntax and expected evidence; memorization is not required.

<code>\$HOME</code>	your home directory, e.g. /home/alice
<code>\$(uname -r)</code>	capture a command's output, e.g. 6.8.0-31-generic
<code>> file</code>	replace a file with stdout (>> appends)
<code>printf</code>	predictable formatted text — safer than echo
<code>test -d PATH</code>	true if a directory exists (-f file, -e exists)
<code>echo \$?</code>	exit status of the last command — 0 means success

Demo: one kernel, two user spaces

Same three questions, two seats: the controller and a throwaway container.

Compare `/etc/os-release`, `uname -r`, and PID 1 on the controller and in a separate demo container.

- `cat /etc/os-release` on both seats — compare the distribution claim
- `uname -r` on both — expect 6.17.0-1026-nvidia on each
- `ps -p 1` on both — which program runs as PID 1?
- Conclusion, stated aloud: the container shares the controller's kernel

Predict: which of the three answers will differ between the two seats?

System boundaries and terminal survival

Lab exercise, 15 minutes: produce the evidence, then explain it.

Create `system-profile.txt` and `path-map.txt` that prove the seat's user space, kernel, architecture, init system, and filesystem anchors.

- 15 minutes
- Prepare/reset only this lab
- Check observable state
- Explain why the result follows

[Open the lab guide for exact tasks, hints, recovery, and checker criteria.](#)

Connect the result to the model

What the lab established.

Distribution and kernel can give different answers because they describe different layers. Next: paths tell us where objects live inside that environment.

- What was declared or expected?
- What runtime state did you observe?
- Which independent evidence closed the claim?

02

Filesystem navigation, objects, and links

How does Linux resolve a pathname into an object?

General Linux · FHS + Linux VFS

Paths resolve from a starting point

A path is a route through one tree; the start point and every component matter.

- Linux presents a single directory tree rooted at /; mounted filesystems join it at directories.
- An absolute path starts at /; a relative path starts at the working directory. Every parent directory needs search (x) permission.

Path resolution checks one component at a time

The final pathname can fail even when the final file mode looks permissive.

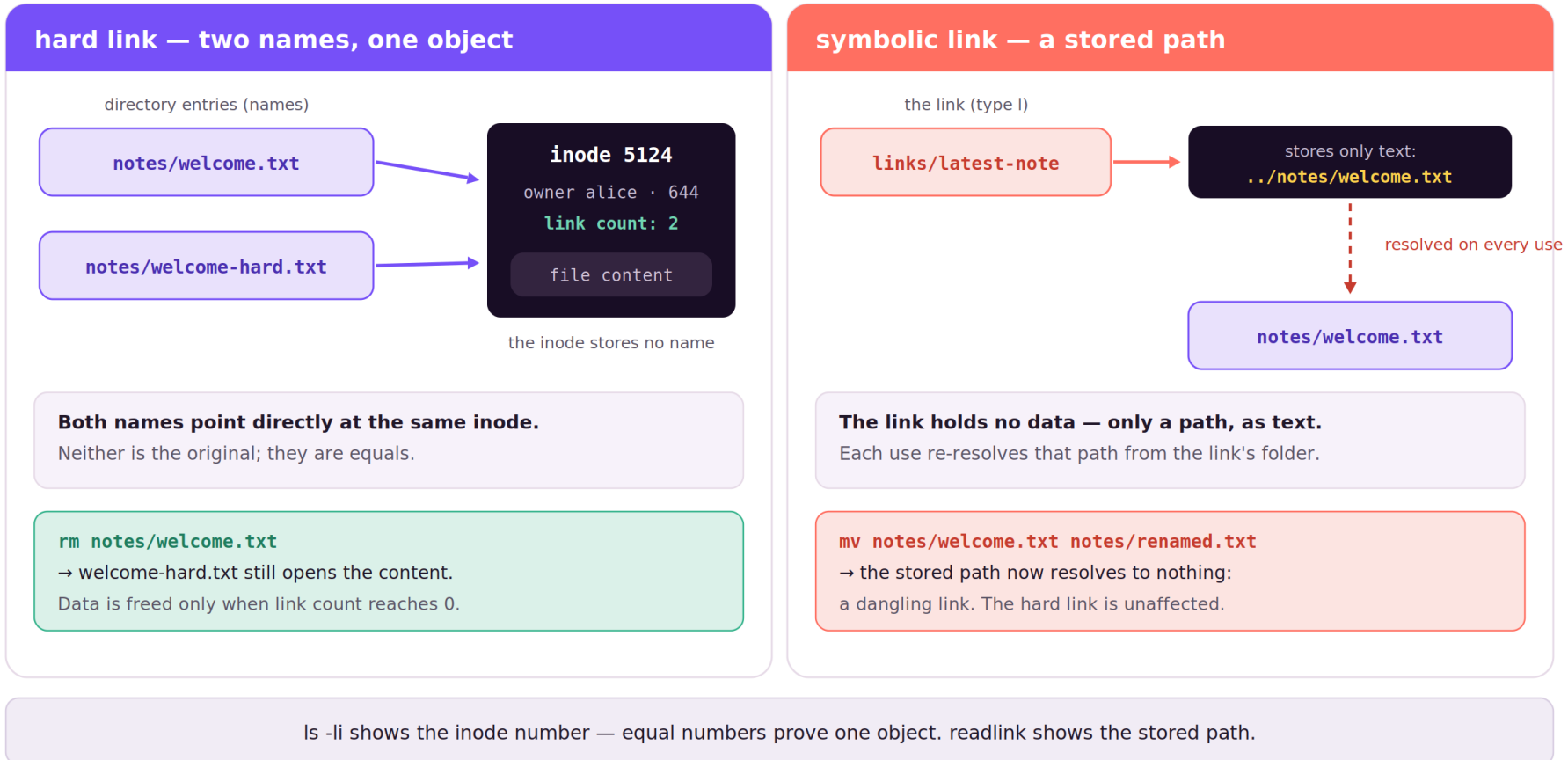
- Choose / or the working directory as the starting point
- Read each directory entry from left to right
- Follow mounts and symbolic links when encountered
- Require search permission on every parent directory

Names, hard links, and symbolic links

First, the object itself: the inode. Names are only labels pointing at it.

- An inode is the filesystem's record for one file — owner, mode, size, timestamps, link count, and where the data lives. Everything except the name.
- A hard link is a second name for the same inode; a symbolic link stores only a path as text, re-resolved on every use.
- Consequence: removing one hard-linked name leaves the content reachable; renaming a symlink's target leaves the link dangling.

Hard link vs symbolic link



A hard link is a second name for the inode; a symlink is a stored path, re-resolved on every use.

Predict link behavior before changing names

```
In notes/welcome.txt notes/welcome-hard.txt  
ln -s ../notes/welcome.txt links/latest-note  
ls -li notes; readlink links/latest-note
```

- ls -li: equal inode numbers prove one object; the link count shows how many names it has.
- readlink prints the stored path text — the only thing a symlink contains.
- Rename the target: the symlink dangles; the hard link is unaffected.
- A relative symlink (../notes/...) resolves from the link's own folder.

These commands are tools for specific questions

The lab guide repeats syntax and expected evidence; memorization is not required.

pwd	the current directory (<code>pwd -P</code> resolves symlinks)
file PATH	guess content type from bytes — a hint, not proof
realpath PATH	the resolved absolute path, symlinks followed
stat PATH	authoritative metadata: inode, links, owner, mode
find ... -printf	list names with fields, e.g. <code>-printf '%p %i'</code>
ls -li	long listing with the inode number in column one

Demo: a 'missing' file that isn't

A 'No such file' error frequently indicates a wrong working directory, not missing data.

Start in the wrong directory, let cat fail, then use pwd, realpath, namei, and stat to correct the coordinate model.

- cat report.txt from the wrong directory — let it fail on screen
- pwd, then realpath report.txt — where am I; what does that name mean here?
- namei -l /home/learner/notes/report.txt — walk every component
- cat with the correct absolute path — the file existed throughout

Predict: is the file missing, or am I in the wrong place — and which command settles it?

Filesystem navigation, objects, and links

Lab exercise, 25 minutes: demonstrate the differing semantics of copy, hard link, and symlink.

Organize ~/inbox, then prove the different semantics of a copy, a hard link, and a relative symbolic link.

- 25 minutes
- Prepare/reset only this lab
- Check observable state
- Explain why the result follows

[Open the lab guide for exact tasks, hints, recovery, and checker criteria.](#)

Connect the result to the model

What the lab established.

Errors such as 'No such file' often reveal a path assumption, not missing data. Next, the shell will turn file and log text into evidence.

- What was declared or expected?
- What runtime state did you observe?
- Which independent evidence closed the claim?

03

Shell execution, streams, editing, and text evidence

How can a pipeline turn records into a reviewable answer?

Bash + GNU text tools · Ubuntu lab

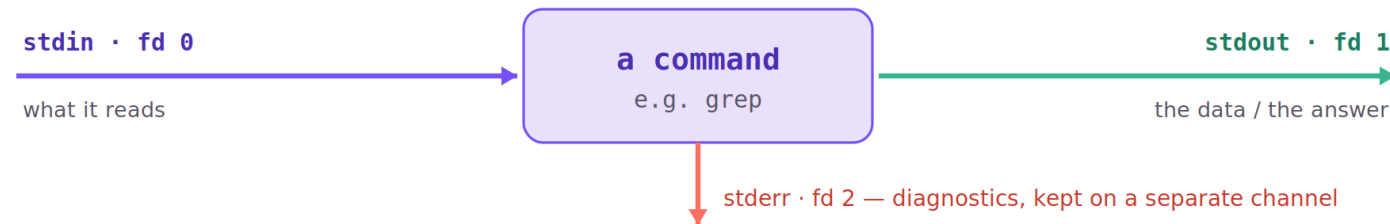
Streams separate data from diagnostics

Every process reads stdin and writes stdout; errors travel separately on stderr.

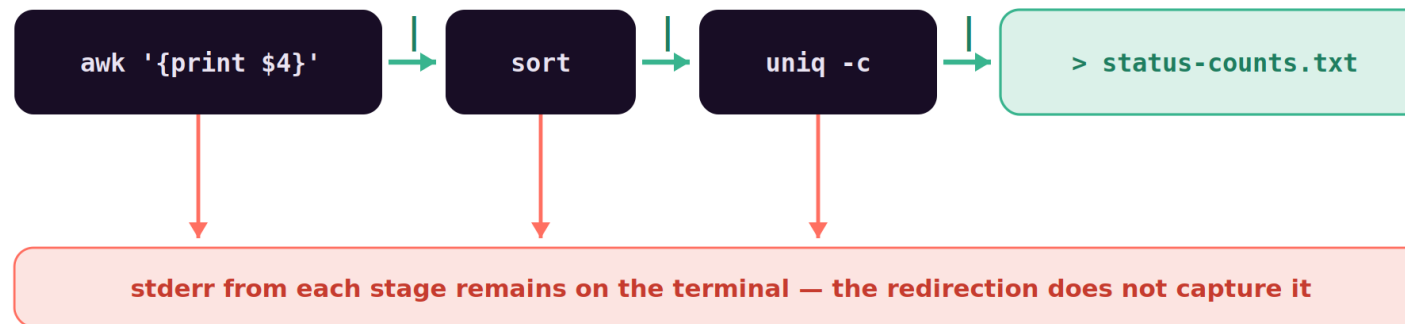
- Redirecting stdout into a file does not capture stderr — diagnostics remain on the terminal.
- The exit status is a third, separate signal: 0 means success, anything else reports failure.

Streams and pipelines

Every process is born with three streams



A pipeline wires one command's stdout into the next one's stdin



A third signal: the exit status. 0 = success. In bash, a pipeline's status is its last command's.

Three streams per process; a pipe joins stdout → stdin, and stderr stays separate.

A pipeline is several processes, not one command

Inspect intermediate output before adding the next stage.

- The shell creates pipes before starting the programs
- Each stage receives the previous stage's standard output
- Standard error remains separate unless explicitly redirected
- The pipeline status normally comes from its last command in Bash

A pipeline is a staged transformation

Each stage receives the previous stage's stdout; build one stage at a time and inspect between stages.

- Typical stages: inspect the record structure, select records, extract a field, group, then save.
- Order matters: sort must precede `uniq -c`, because `uniq` only counts adjacent identical lines.

Make every pipeline stage explainable

```
awk '{print $4}' access.log \  
| sort | uniq -c \  
| awk '{print $2, $1}' > status-counts.txt
```

- `awk '{print $4}'` pulls one field: the status code, one per line.
- `sort` clusters equal codes so `uniq -c` can count each run.
- `uniq -c` without `sort` miscounts; it compares adjacent lines only.
- Redirect once at the very end; inspect each stage's output first.

These commands are tools for specific questions

The lab guide repeats syntax and expected evidence; memorization is not required.

nano FILE	edit in-terminal — Ctrl+O save, Ctrl+X exit
grep PATTERN FILE	keep matching lines (-v drops, -c counts)
awk	extract / reshape fields: \$1 first, \$NF last
sort	order lines (-n numeric, -k by column)
uniq -c	count adjacent duplicates; requires sorted input
wc -l	count lines — a quick sanity check on output

Demo: a pipeline built one stage at a time

Build the pipeline incrementally: one stage, inspect, then the next.

Inspect a synthetic authentication log, predict the needed fields, build the pipeline one stage at a time, then save and count the evidence.

- `head auth.log` — inspect the record structure before touching it
- `awk '{print $3}' auth.log | head` — extract one field, inspect again
- `... | sort | uniq -c` — group first, then count adjacent duplicates
- `> failure-counts.txt`, then `cat` it — save, and verify what was saved

Predict: what would the counts look like if sort were skipped?

Shell execution, streams, editing, and text evidence

Lab exercise, 20 minutes: reduce raw logs to evidence files a colleague can review.

Edit `app.conf` and reduce application and access logs into small evidence files another operator can review.

- 20 minutes
- Prepare/reset only this lab
- Check observable state
- Explain why the result follows

[Open the lab guide for exact tasks, hints, recovery, and checker criteria.](#)

Connect the result to the model

What the lab established.

A pipeline is trustworthy when each stage has a stated input and output. Next, package tools will provide several independent views of software state.

- What was declared or expected?
- What runtime state did you observe?
- Which independent evidence closed the claim?

04

Packages and software provenance

What proves that software came from the supported package system?

Ubuntu 24.04 · APT + dpkg

Package tools expose different states

The repository, the local index, and the installed database are three different things.

- A repository publishes signed packages; 'apt update' refreshes only the local index and installs nothing.
- APT resolves and fetches from repositories; dpkg records what is installed and which files each package owns.

Software state has several independent layers

A command that runs is not automatically a supported package installation.

- Repository metadata says what is available
- The package database says what is registered as installed
- The shell decides which executable path it would run
- The executable's behavior is a final, separate observation

Provenance is a chain of five claims

Each claim has its own command; none of them follows from the others.

- A candidate exists → the package is registered installed → an executable is on PATH → that file belongs to the package.
- The fifth claim — the program behaves correctly — is a separate observation, tested last.

Verify software through independent views

```
apt-cache policy tree
sudo apt install tree
dpkg-query -W -f='${Status}\n' tree
command -v tree
dpkg-query -S "$(command -v tree)"
```

- apt-cache policy: the candidate version, before you install.
- command -v finds it on PATH; that is discovery, not ownership.
- dpkg-query -S names the package that owns that exact file.
- Only after all four checks does running tree test real behaviour.

These commands are tools for specific questions

The lab guide repeats syntax and expected evidence; memorization is not required.

<code>apt-cache policy NAME</code>	installed vs candidate version, and the repo
<code>sudo apt update</code>	refresh the package index only — installs nothing
<code>sudo apt install NAME</code>	resolve dependencies and install from the repo
<code>dpkg-query -W</code>	registered installed? prints name and version
<code>dpkg-query -S PATH</code>	which installed package owns this exact file
<code>apt-get -s install X</code>	simulate — show what would change, do nothing

Demo: proving where jq comes from

Trace one tool, jq, through the complete provenance chain.

Trace jq from repository candidate to installed state, executable path, package ownership, and JSON behavior.

- `apt-cache policy jq` — a candidate exists; note the repository
- `sudo apt install jq`, then `dpkg-query -W jq` — registered as installed
- `command -v jq`, then `dpkg-query -S` on that path — discovered and owned
- `echo '{}'` | `jq .` — behaviour, tested last

Predict: which of the five provenance claims does each command close?

Packages and software provenance

Lab exercise, 20 minutes: install tree and document every layer of its provenance.

Install tree from the configured Ubuntu repositories and create evidence for candidate, state, path, ownership, and behavior.

- 20 minutes
- Prepare/reset only this lab
- Check observable state
- Explain why the result follows

[Open the lab guide for exact tasks, hints, recovery, and checker criteria.](#)

Connect the result to the model

What the lab established.

Package state is a chain of claims, not a single command result. Next, identity and permissions determine who may change those files and directories.

- What was declared or expected?
- What runtime state did you observe?
- Which independent evidence closed the claim?

05

Users, groups, permissions, and shared directories

Why does one identity gain access while another is denied?

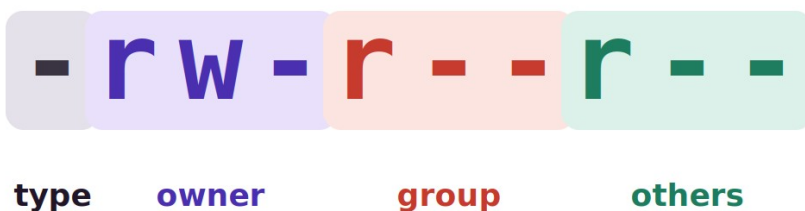
General Linux DAC · Ubuntu sudo

Access checks begin with process identity

The kernel evaluates numeric identities against exactly one permission class.

- A process carries an effective user and a group set, captured at login; the kernel checks these numbers, not names.
- For mode bits exactly one class applies — owner, else matching group, else other — never a combination.

Reading an ls -l line — the permission bits



Each of the 9 slots is read / write / execute

- 1st char: file type: - file d dir l symlink
- owner rw-: alice may read + write, not execute
- group r--: members of devs may read only
- others r--: everyone else may read only

Turn each slot into a number — add r=4, w=2, x=1

rw- r-- r-- → 6 4 4 = 644

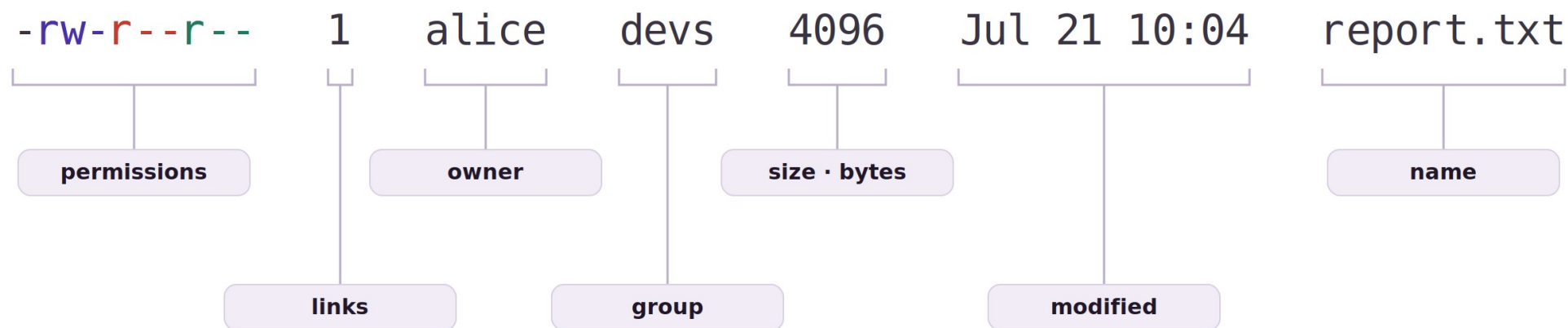
r = 4 w = 2 x = 1

a missing letter counts as 0

another: rwxr-x--- → 7 5 0 (owner rwx=7, group r-x=5, others ---=0)

First char is the type; then owner / group / others, each read-write-execute — then the octal.

Reading an ls -l line — the whole line



The same line, field by field: permissions, links, owner, group, size, time, name.

Create users and groups

```
sudo groupadd developers  
sudo useradd -m -G developers alice  
sudo usermod -aG developers bob  
id alice
```

- groupadd: create the group first
- useradd -m: new user + home dir; -G adds extra groups
- usermod -aG: append a group (-a, or you replace them all)
- id alice: confirm uid, primary group, and membership

setuid, setgid, and the sticky bit

setuid	octal 4000	setgid	octal 2000	sticky	octal 1000
- r w s r - x r - x the special bit					
executes with the file owner's identity, not yours e.g. /usr/bin/passwd runs as root so you can set your password					
d r w x r w s - - - the special bit					
on a directory: new files inherit the directory's group on a file: runs as the file's group — used for team folders					
d r w x r w x r w t the special bit					
in a shared directory, only a file's owner may delete it e.g. /tmp — everyone writes, nobody deletes another's files					

lowercase s / t = the execute bit is set too · UPPERCASE S / T = bit is set but NOT executable

Three extra bits, each shown by one character in the ls -l string.

Directory permissions govern path operations

Reaching a file requires search permission on every directory along the path.

- On a directory: r lists names, w creates or removes names, x traverses it.
- setgid makes new files inherit the directory's group; the sticky bit restricts deletion in shared directories.

Permission diagnosis follows a fixed decision order

Identify which of the four checks failed before changing any mode.

- Identify the identity — `id` shows the uid, gid, and group set the kernel uses.
- Check search (x) on every parent — `namei -l` walks the whole path.
- Pick the one class that applies — owner, else group, else other (never combined).
- Test the exact operation as that identity — `sudo -u alice test -w DIR`.

Repair from identity outward

```
id alice
namei -l /srv/team-share/report.txt
sudo chown root:webteam /srv/team-share
sudo chmod 2770 /srv/team-share
sudo -u alice test -w /srv/team-share
```

- id: the uid, gid, and groups the kernel checks — not the name.
- namei -l walks each parent and shows where x is missing.
- sudo -u alice evaluates the policy as alice; testing as your own identity is not informative.
- chmod 2770 = setgid + rwx owner + rwx group + --- other.
- The leading 2 is setgid: new files inherit the directory's group.

These commands are tools for specific questions

The lab guide repeats syntax and expected evidence; memorization is not required.

<code>id USER</code>	uid, primary group, and supplementary groups
<code>namei -l PATH</code>	every path component with its mode and owner
<code>stat -c ... PATH</code>	owner, group, mode, e.g. <code>stat -c '%U %G %a'</code>
<code>sudo -u USER COMMAND</code>	run a check as another identity — the decisive test
<code>chmod 2770 DIR</code>	setgid + rwx owner/group, nothing for others
<code>chown u:g PATH</code>	set owner and group (<code>chgrp</code> sets group only)

Demo: diagnose before chmod

Diagnose before modifying: two identities, every path component, then the minimal change.

Repair /srv/demo-share for demoops after inspecting two users, every path component, group ownership, and directory semantics.

- `id demoops; id outsider` — the numeric identities the kernel will check
- `namei -l /srv/demo-share/report.txt` — find the failing component
- `sudo chown root:demoops + chmod 2770` — one minimal, named change
- `sudo -u demoops test -w ...; sudo -u outsider test -r ...` — both verdicts

Predict: which check fails first — the class selection, or search permission on a parent?

Users, groups, permissions, and shared directories

Lab exercise, 30 minutes: grant the team access, keep others denied, and verify both.

Repair `/srv/team-share` for `webteam` so collaborators can create and update files while an outsider remains denied.

- 30 minutes
- Prepare/reset only this lab
- Check observable state
- Explain why the result follows

[Open the lab guide for exact tasks, hints, recovery, and checker criteria.](#)

Connect the result to the model

What the lab established.

Least privilege comes from matching identity, path traversal, and the intended operation. Next, SSH will authenticate both ends of a remote session.

- What was declared or expected?
- What runtime state did you observe?
- Which independent evidence closed the claim?

06

SSH transport, host trust, and user authentication

How does an SSH client know the server—and how does the server know the user?

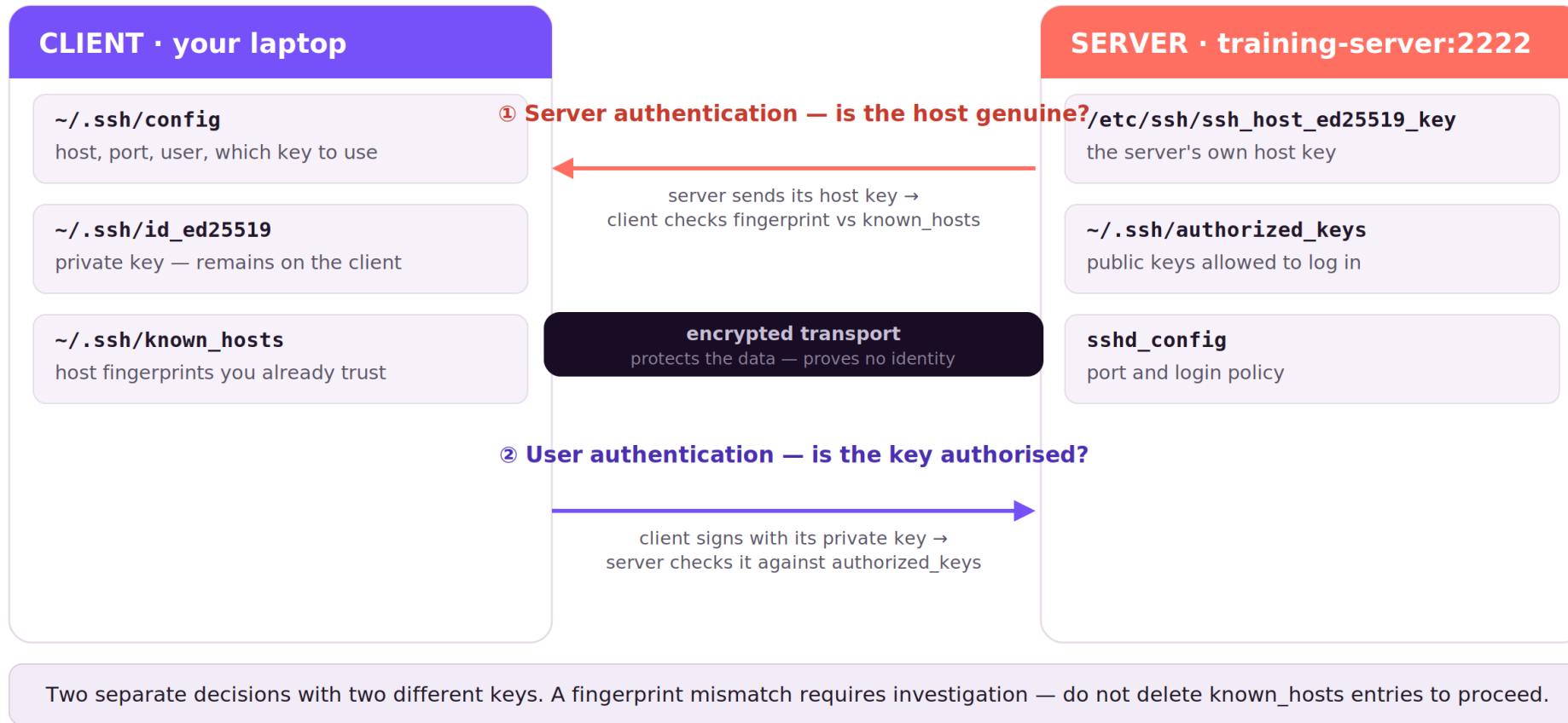
OpenSSH · Ubuntu lab endpoint

SSH authenticates both ends

Encryption alone establishes no identity; each end must prove who it is.

- The server proves itself with its host key; the client compares its fingerprint against `known_hosts`.
- A first-time fingerprint must be compared through an independent, trusted channel before it is accepted.

SSH proves two identities



Transport encryption alone is insufficient; these two verifications establish identity.

An SSH connection establishes trust in stages

A failure at one stage should not be “fixed” by weakening another.

- Reach the correct host and TCP port
- Verify the server host-key fingerprint independently
- Offer proof from the client private key
- Let the server match the authorized public key and account policy

User authentication is a separate key pair

The private key proves possession; the server authorises its public counterpart.

- The private key remains on the client; the matching public key is listed in the server's `authorized_keys`.
- OpenSSH verifies ownership and permissions on these files — an over-permissive private key is refused.

Verify host trust before user trust

```
ssh-keyscan -p 2222 training-server > host.scan  
ssh-keygen -lf host.scan -E sha256  
ssh -o BatchMode=yes training-server
```

- ssh-keyscan grabs the host key — collecting is not trusting.
- ssh-keygen -lf prints the SHA256 fingerprint to compare out-of-band.
- BatchMode=yes fails loudly instead of dropping to a password.
- Match the fingerprint before it is written to known_hosts.

These commands are tools for specific questions

The lab guide repeats syntax and expected evidence; memorization is not required.

<code>ssh-keygen -t ed25519</code>	make a modern key pair — private key stays put
<code>ssh-keyscan</code>	fetch a server's host key over the network
<code>ssh-keygen -lf</code>	show a key's SHA256 fingerprint
<code>ssh -G HOST</code>	effective client config: which key, port, user
<code>BatchMode=yes</code>	no prompts — fail instead of asking for a password
<code>chmod 600 ~/.ssh/id_ed25519</code>	a group/world-readable key is refused

Demo: trusting a host, then proving a user

Verify the host first: compare the fingerprint out-of-band, then record it and authenticate by key.

Compare the demo endpoint fingerprint through an independent source, pin it, and prove key authentication on port 2207.

- `ssh-keyscan -p 2207 demo-host > scan` — collect the key; no trust yet
- `ssh-keygen -lf scan` — the fingerprint, compared against the posted value
- `ssh -o BatchMode=yes -p 2207 demo-host` — key login, no password fallback
- On a mismatch: stop. `known_hosts` is never deleted to make progress

Predict: what does a matching fingerprint prove — identity, or only consistency?

SSH transport, host trust, and user authentication

Lab exercise, 25 minutes: build a strict SSH profile and verify key-only login.

Create a strict client profile for training-server on port 2222, verify its host key, authorize a dedicated user key, and prove BatchMode login.

- 25 minutes
- Prepare/reset only this lab
- Check observable state
- Explain why the result follows

[Open the lab guide for exact tasks, hints, recovery, and checker criteria.](#)

Connect the result to the model

What the lab established.

A successful SSH session rests on two explicit trust decisions. Next, logs will help reconstruct what happened when a service or login fails.

- What was declared or expected?
- What runtime state did you observe?
- Which independent evidence closed the claim?

07

Logs, journal scope, and causal events

Which event explains the failure rather than merely reporting its consequence?

systemd journal · Ubuntu 24.04

Logs are bounded event evidence

Scope the window first — unit, boot, time — then read chronology before filtering.

- The journal stores structured fields: time, unit, priority, message.
- A premature keyword search discards the chronology that explains the failure.

A useful incident window preserves causality

Narrow scope reduces noise without destroying chronology.

- Select the affected unit and boot
- Bound the time around the reported symptom
- Read the sequence before filtering for keywords
- Separate the first cause from later failure summaries

State summaries and causal events differ

`systemctl show` reports where the unit ended up; `journalctl` shows how it got there.

- `ActiveState`, `Result`, and `ExecMainStatus` summarise the outcome — they do not explain it.
- The final 'failed' entry is usually a consequence; the cause is the earliest relevant record in the window.

Narrow evidence without losing chronology

```
systemctl show noisy.service \
-p ActiveState -p Result -p ExecMainStatus
journalctl -u noisy.service -b \
--since "-10 min" -o short-iso --no-pager
```

- show -p gives ActiveState / Result / ExecMainStatus — the symptom.
- -u UNIT and -b scope to one service on this boot before you read.
- --since '-10 min' bounds time so anyone can repeat the query.
- Read top to bottom: the first error usually causes the later 'failed'.

These commands are tools for specific questions

The lab guide repeats syntax and expected evidence; memorization is not required.

<code>journalctl -u UNIT</code>	one service's log, e.g. <code>journalctl -u ssh</code>
<code>-b</code>	this boot only (<code>-b -1</code> = previous boot)
<code>--since/--until</code>	bound time: <code>--since '09:00' --until '09:10'</code>
<code>-p PRIORITY</code>	severity: err, warning, info — after scoping
<code>--no-pager</code>	plain output, good for scripts and copy-paste
<code>journalctl -xe</code>	recent entries with hints — a fast first look

Demo: the line that says 'failed' is not the cause

A service exits 17: we find the line that caused it, not the 'failed' line after it.

Start a service that records STALE_CACHE and exits 17; identify the causal record before the later failed-state messages.

- `systemctl start noisy-cache; systemctl status` — the symptom summary
- `systemctl show -p Result -p ExecMainStatus` — exit code 17, recorded
- `journalctl -u noisy-cache -b --since '-5 min'` — read the window in order
- The earliest causal record: STALE_CACHE — above the later 'failed' lines

Predict: will the cause sit near the top of the window, or the bottom?

Logs, journal scope, and causal events

Lab exercise, 20 minutes: preserve the evidence and explain exit code 42.

Investigate a disk-threshold incident, preserve unit-scoped journal evidence, and explain why exit code 42 occurred.

- 20 minutes
- Prepare/reset only this lab
- Check observable state
- Explain why the result follows

[Open the lab guide for exact tasks, hints, recovery, and checker criteria.](#)

Connect the result to the model

What the lab established.

The journal connects state to a causal sequence. Next, systemd declarations and relationships explain what the manager attempted to do.

- What was declared or expected?
- What runtime state did you observe?
- Which independent evidence closed the claim?

08

systemd declarations, dependencies, ordering, and diagnosis

How does systemd turn a unit declaration into runtime behavior?

systemd · Ubuntu 24.04

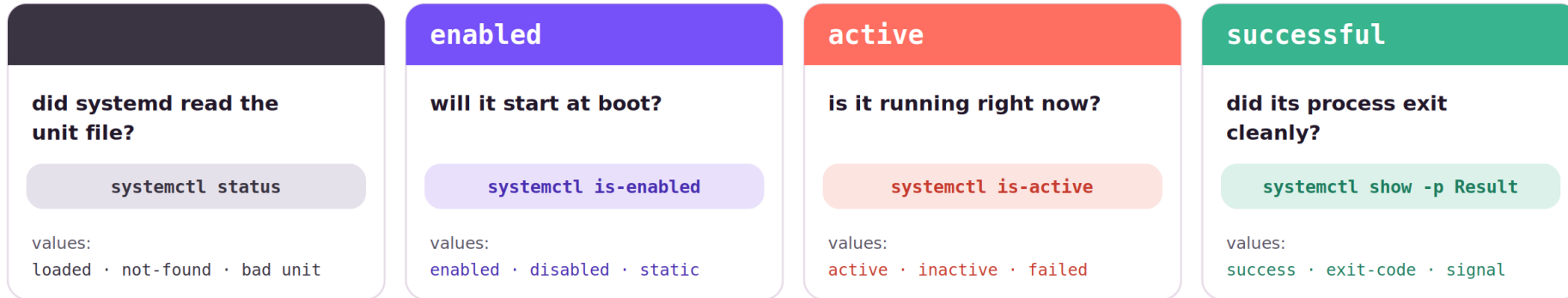
A unit declaration is not runtime state

The manager tries to reach the declared state and records every transition.

- A unit file declares command, identity, working directory, dependencies, and restart policy.
- Loaded, enabled, active, and successful are four independent facts; 'status' summarises symptoms only.

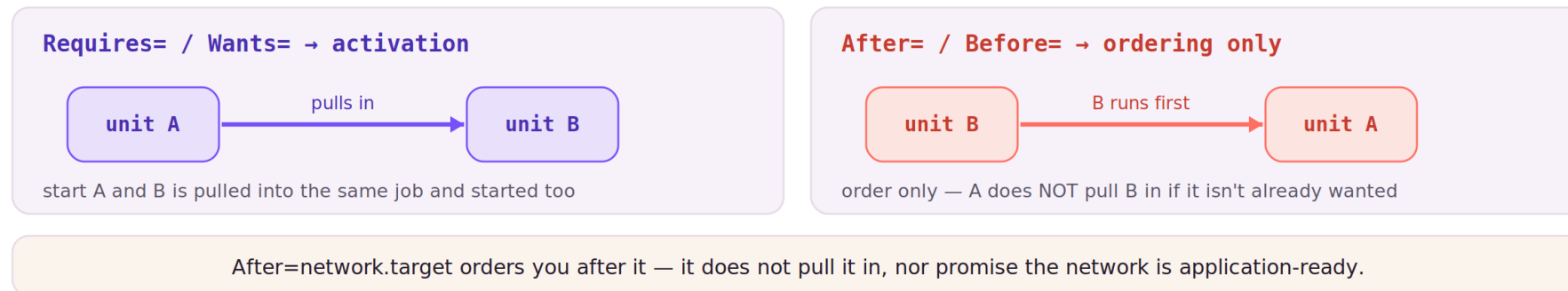
Four states, and needs vs order

Four independent state dimensions — verify each separately



e.g. a unit can be enabled but not active, or active but not successful — they don't imply each other.

Two distinct relationships: activation and ordering



Loaded, enabled, active, successful are independent — and Requires is not After.

Requirement and ordering are separate relations

Requires= and Wants= select units; After= and Before= only sequence them.

- Ordering never pulls a missing unit into the transaction.
- After=network.target sequences startup; it does not make the network application-ready.

systemd builds a transaction before starting processes

Requires= does not mean “run first”; After= does not mean “start it”.

- Activation relationships decide which units join the transaction
- Ordering relationships decide which job runs before another
- Conditions and prerequisites can stop execution before ExecStart
- Every transition leaves manager and application evidence

Read diagnosis from declaration to behavior

```
systemctl status demo-api --no-pager  
journalctl -u demo-api -b --no-pager  
systemctl cat demo-api  
systemctl show demo-api -p Wants -p Requires -p After
```

- status: the symptom line plus a few recent log entries.
- journalctl -u: the transitions and the real failure message.
- systemctl cat: the effective unit, drop-ins included.
- show -p Requires -p After: the real dependency graph, normalised.

Create a minimal service of your own

```
# /etc/systemd/system/hello.service
[Unit]
Description=Hello web service
After=network.target

[Service]
ExecStart=/usr/bin/python3 -m http.server 8080
WorkingDirectory=/srv/hello
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

- ① `sudo systemctl daemon-reload`
- ② `sudo systemctl enable --now hello`
- ③ `systemctl status hello · curl :8080`

These commands are tools for specific questions

The lab guide repeats syntax and expected evidence; memorization is not required.

status/is-active

runtime state now: active / failed / inactive

is-enabled

boot policy: enabled / disabled / static

cat/show

effective unit text; show = normalised key=value

daemon-reload

re-read unit files after you change them

restart

new process from the effective definition

systemctl list-dependencies U

what this unit pulls in, as a tree

Demo: the unit is fine; its world is not

The unit is fine, its prerequisite is missing. We add only that, then re-verify.

Diagnose demo-api.service whose required EnvironmentFile is missing; create only the missing prerequisite, restart, then verify its endpoint.

- `systemctl status demo-api` — failed; read the hint in the summary
- `journalctl -u demo-api -b` — the actual error: EnvironmentFile missing
- `systemctl cat demo-api` — the declaration names the missing file
- Create the file → restart → curl the endpoint. The unit was never edited

Predict: edit the unit, or satisfy its declaration — which is safer, and why?

systemd declarations, dependencies, ordering, and diagnosis

Lab exercise, 25 minutes: repair the service without editing its unit; verify all four layers.

Repair `course-web.service` from its CHDIR failure without editing the supplied unit, then prove state, enablement, socket, and HTTP behavior.

- 25 minutes
- Prepare/reset only this lab
- Check observable state
- Explain why the result follows

[Open the lab guide for exact tasks, hints, recovery, and checker criteria.](#)

Connect the result to the model

What the lab established.

The safest repair satisfies the declaration's missing prerequisite rather than forking it. Next, processes and sockets expose the running implementation.

- What was declared or expected?
- What runtime state did you observe?
- Which independent evidence closed the claim?

09

Processes, sockets, and client-visible behavior

How do unit state, process state, socket state, and useful behavior connect?

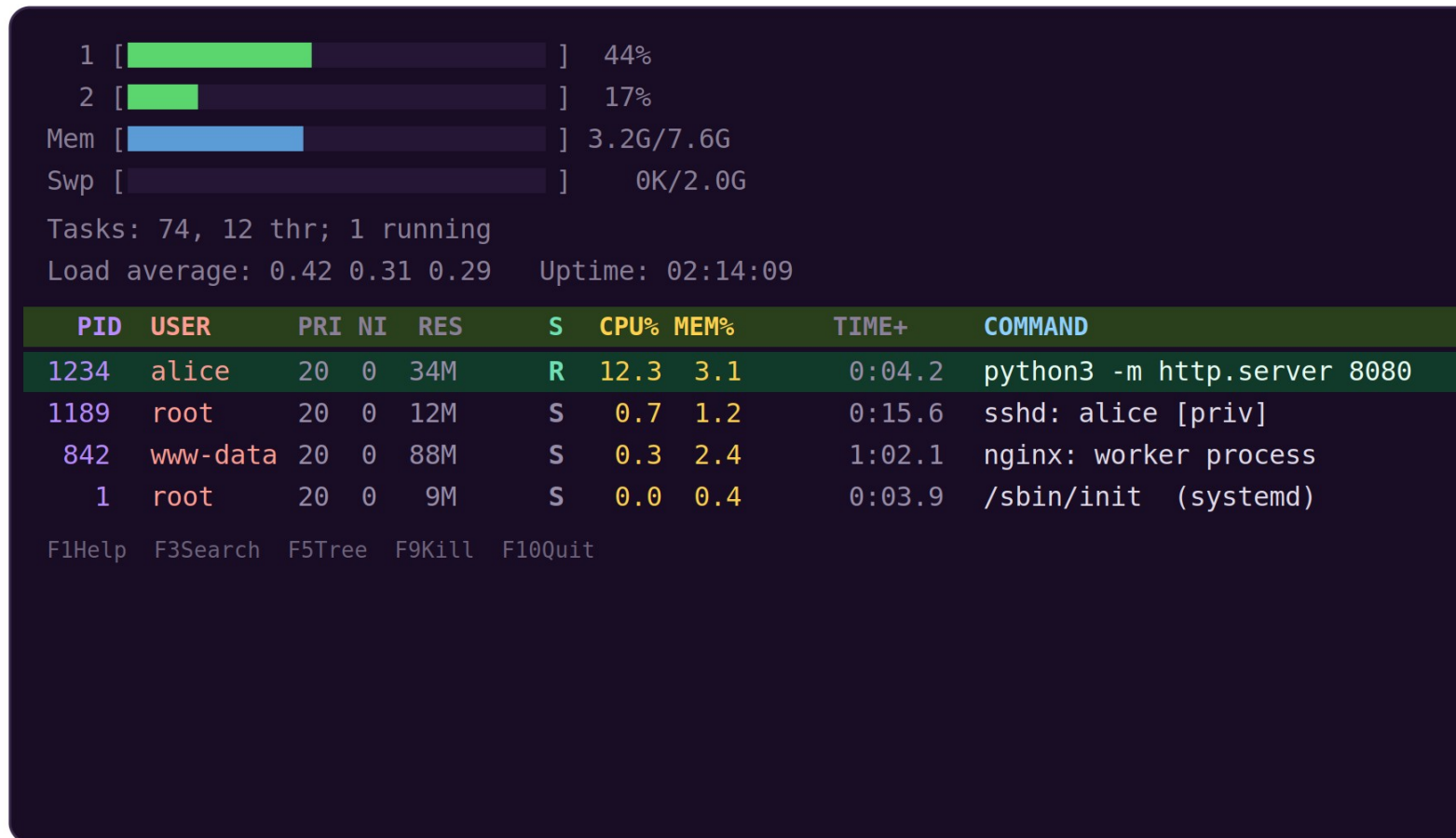
General Linux processes + iproute2 + systemd

A process is a running program with context

The executable on disk is only a template; the process is its live instantiation.

- A process has a PID, a parent, an identity, memory, and open file descriptors.
- systemd's MainPID ties a service to one process; ps and /proc/PID expose its context.

What htop shows you about a process



Reading one process

PID

one number identifying a running program (temporary — it changes)

USER

the effective identity the kernel checks for every permission

S

state: R = running now,
S = sleeping / waiting

CPU% MEM%

the CPU time and memory the kernel is arbitrating for it

COMMAND

the program plus its arguments — as invoked — its exact arguments

PID 1 = systemd is the ancestor of every service you'll manage.

Each row is an identity, a state, and allocated resources; run htop to observe this live.

A listener is not correct behaviour

Accepting connections and answering correctly are different claims.

- ss shows that a process listens at a port; only a real client such as curl verifies the answer.
- Stop managed processes through systemd, so the manager's state remains accurate.

A service claim crosses four observable layers

Each layer can succeed while the next one still fails.

- Manager state connects the unit to MainPID
- Process evidence shows identity and command context
- Socket evidence shows whether a transport endpoint is listening
- A real client proves application-visible behavior

Trace one service through four layers

```
pid=$(systemctl show course-probe -p MainPID --value)
ps -o pid,ppid,user,stat,cmd -p "$pid"
sudo ss -ltnp 'sport = :9099'
curl --fail http://127.0.0.1:9099/
```

- MainPID is the one process systemd owns for this service.
- ss -ltnp maps the port to a listening process (sudo for the name).
- curl --fail is the only line that speaks for a real client.
- curl --fail: an HTTP error becomes a non-zero exit, not a nice page.

These commands are tools for specific questions

The lab guide repeats syntax and expected evidence; memorization is not required.

`systemctl show -p MainPID`

the service's main process id

`ps -p PID`

one process: user, state, command (-o picks columns)

`/proc/PID/cmdline`

exact argv, NUL-separated — supporting evidence

`ss -ltnp`

listening TCP sockets + owning process (l t n p)

`curl --fail URL`

client test; --fail makes an HTTP error a non-zero exit

`htop`

live processes, CPU/mem, tree, search — F9 to signal

Demo: four layers, one service

Trace one service through all four layers, then stop it and confirm the socket is released.

Trace demo-echo.service through MainPID, ps, command line, port 9191, and HTTP; stop it through systemd and prove the socket disappears.

- `systemctl show demo-echo -p MainPID → ps -p that PID` — manager to process
- `sudo ss -ltnp 'sport = :9191'` — the listener, owned by that process
- `curl --fail localhost:9191` — the client's verdict on behaviour
- `systemctl stop demo-echo → ss again` — the socket is gone

Predict: after the stop, what does each of the four layers report?

Processes, sockets, and client-visible behavior

Lab exercise, 15 minutes: trace a service end to end, then verify its removal.

Trace `course-probe.service` through unit, PID, user, command line, port 9099, and HTTP, then stop it and prove the listener is gone.

- 15 minutes
- Prepare/reset only this lab
- Check observable state
- Explain why the result follows

[Open the lab guide for exact tasks, hints, recovery, and checker criteria.](#)

Connect the result to the model

What the lab established.

No single interface proves the whole service. Next, unit layering will make a configuration change without editing the supplied fragment.

- What was declared or expected?
- What runtime state did you observe?
- Which independent evidence closed the claim?

10

Unit layering, drop-ins, reloads, and durable changes

How can an administrator override a unit without forking the vendor file?

systemd unit layering · Ubuntu 24.04

Effective configuration is assembled from layers

Vendor fragments, administrator units, and drop-ins combine by load-path precedence.

- A drop-in records a narrow override while the shipped fragment stays intact for updates and comparison.
- Some list-valued directives require an explicit reset before new entries are added.

A drop-in changes one layer of effective configuration

Plan removal of the drop-in as the rollback path.

- The supplied fragment remains intact
- Administrator drop-ins load with higher precedence
- daemon-reload updates the manager's definition
- restart creates a new process; behavior verifies the result

Reloading definitions is not restarting

daemon-reload re-reads unit files; only restart applies them to a process.

- After daemon-reload the running process is unchanged; restart creates a new execution.
- Enabled, active, reloaded, restarted, and observed behaviour are verified separately.

Close the loop from layer to behavior

```
systemctl show course-banner \
  -p FragmentPath -p DropInPaths -p Environment
sudo systemctl daemon-reload
sudo systemctl restart course-banner
cat /run/course-banner.txt
```

- FragmentPath = shipped file; DropInPaths = your overrides, in order.
- daemon-reload updates systemd's view — the process is still old.
- restart makes the new process; then check what it produced.
- The regenerated /run file is the proof — not the edit itself.

These commands are tools for specific questions

The lab guide repeats syntax and expected evidence; memorization is not required.

<code>systemctl cat UNIT</code>	shipped unit plus every drop-in, in load order
<code>systemctl edit UNIT</code>	create an override drop-in the safe way
<code>FragmentPath/DropInPaths</code>	where each setting comes from (provenance)
<code>daemon-reload</code>	re-read definitions after editing
<code>restart + behavior check</code>	apply, then verify the output actually changed
<code>systemctl revert U</code>	drop all overrides, back to the shipped unit

Demo: a drop-in, a reload, a restart

Change one value with a drop-in, then verify the running process observed it.

Override DEMO_LOG_LEVEL for demo-banner.service, compare effective configuration, reload, restart, and verify generated output.

- `systemctl show demo-banner -p Environment` — the value before
- `systemctl edit demo-banner` — write the DEMO_LOG_LEVEL override
- `daemon-reload` → show again: the manager knows; the output file is unchanged
- `restart` → the regenerated file finally reflects the new value

Predict: what changes after the reload but before the restart?

Unit layering, drop-ins, reloads, and durable changes

Lab exercise, 15 minutes: make a durable change via drop-in, with a rollback path.

Set `COURSE_MODE=production` with a drop-in, reload definitions, restart the service, and prove effective configuration and regenerated behavior.

- 15 minutes
- Prepare/reset only this lab
- Check observable state
- Explain why the result follows

[Open the lab guide for exact tasks, hints, recovery, and checker criteria.](#)

Connect the result to the model

What the lab established.

A durable change is complete only when provenance, runtime state, and behavior agree, and rollback is understood.

- What was declared or expected?
- What runtime state did you observe?
- Which independent evidence closed the claim?

A safe operator closes every change with evidence

Locate context → predict state → act narrowly → observe output and events → verify behavior independently.

- Declaration is not runtime state
- Runtime state is not useful behavior
- One successful command is not the whole claim

Explain the chain when you are blocked

A useful help request contains enough unchanged evidence for someone else to reason with you.

- Which user, host, and working directory?
- What exact target state did you inspect?
- What command and unchanged output can be repeated?
- What changed, and what independent check followed?

Protect the 50/50 balance when adapting delivery

If the room schedule changes, shorten examples and discussion evenly; keep all ten labs, verification steps, and debriefs.

- 50% guided teaching
- 50% hands-on practice
- Do not trade learner verification for lecture time

Practise the loop on a reset incident

Reset one lab, solve it without copying the prior command sequence, and explain the evidence aloud.

- Rebuild the SSH profile from the trust model
- Explain 640 and 2770 without a lookup
- Trace a service from unit to client behavior

Primary manuals define the claims

The course claim ledger maps every module and lab to authoritative documentation.

- GNU Bash Reference Manual
- Ubuntu Server documentation
- systemd manuals
- OpenSSH manuals
- Linux man-pages and FHS
- Incus instance documentation

Operate the system. Trust the evidence.

You do not need to remember every command. You need to know what question to ask, what changed, and what would prove it.
